

Name: Monisha Raghini Chelle

RollNo:2520030537

Section-6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

21.To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14      Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                          OSSP-Practical  example.txt             hello       moni    sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello      hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c    hello:Zone.Identifier    hello_shortcut  skill        virtual_address
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the integrated shell

Run:

nano integrated_shell.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano integrated_shell.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 integrated_shell.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_ARGS 20
#define MAX_JOBS 10
#define INPUT_SIZE 256

typedef struct {
    int id;
    pid_t pid;
    char command[INPUT_SIZE];
    int active;
} Job;

static Job jobs[MAX_JOBS];
static int next_id = 1;

static void update_jobs(void)
{
    int status;
    pid_t pid;

    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        for (int i = 0; i < MAX_JOBS; i++) {
            if (jobs[i].active && jobs[i].pid == pid) {
                printf("\n[%d] Job completed: %s\n",
                    jobs[i].id, jobs[i].command);
                jobs[i].active = 0;
                break;
            }
        }
    }
}

static void show_jobs(void)
{
    int found = 0;
    update_jobs();
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^Y Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line M-F Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 integrated_shell.c *

printf("\nID\tPID\tCOMMAND\n");

for (int i = 0; i < MAX_JOBS; i++) {
    if (jobs[i].active) {
        printf("%d\t%d\t%s\n",
               jobs[i].id, (long)jobs[i].pid,
               jobs[i].command);
        found = 1;
    }
}

if (!found)
    printf("No active background jobs.\n");
}

static int tokenize(char *line, char *args[])
{
    int count = 0;
    char *token = strtok(line, " \t");

    while (token != NULL) {
        if (count >= MAX_ARGS - 1) {
            fprintf(stderr, "Error: Too many arguments.\n");
            return -1;
        }

        args[count++] = token;
        token = strtok(NULL, " \t");
    }

    args[count] = NULL;
    return count;
}

static void execute_command(char *input)
{
    char original[INPUT_SIZE];
    char *args[MAX_ARGS];

    snprintf(original, sizeof(original), "%s", input);

    int count = tokenize(input, args);

    if (count <= 0)
```

Help	Write Out	Where Is	Cut	Execute	Location	Undo
Exit	Read File	Replace	Paste	Justify	Go To Line	Redo

```
return;

int background = 0;

if (strcmp(args[count - 1], "&") == 0) {
    background = 1;
    args[--count] = NULL;

    if (count == 0) {
        fprintf(stderr, "Error: Missing command before &.\n");
        return;
    }
}

if (strcmp(args[0], "pwd") == 0) {
    char cwd[INPUT_SIZE];

    if (count != 1 || background) {
        fprintf(stderr, "Usage: pwd\n");
        return;
    }

    if (getcwd(cwd, sizeof(cwd)))
        printf("%s\n", cwd);
    else
        perror("getcwd");

    return;
}

if (strcmp(args[0], "cd") == 0) {
    if (count != 2 || background) {
        fprintf(stderr, "Usage: cd DIRECTORY\n");
        return;
    }

    if (chdir(args[1]) == -1)
        perror("cd");

    return;
}

if (strcmp(args[0], "jobs") == 0) {
    if (count != 1 || background) {
        fprintf(stderr, "Usage: jobs\n");
    }
}
```

```
fprintf(stderr, "Usage: jobs\n");
return;
}

show_jobs();
return;
}

if (background) {
    int free_slot = -1;

    update_jobs();

    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active) {
            free_slot = i;
            break;
        }
    }

    if (free_slot == -1) {
        fprintf(stderr, "Error: Job table full.\n");
        return;
    }

    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        execvp(args[0], args);
        perror(args[0]);
        _exit(127);
    }

    jobs[free_slot].id = next_id++;
    jobs[free_slot].pid = pid;
    jobs[free_slot].active = 1;

    snprintf(jobs[free_slot].command,
             sizeof(jobs[free_slot].command),
             "%s", original);
}
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 integrated_shell.c *

    printf("[%d] Background PID=%ld\n",
           jobs[free_slot].id, (long)pid);

    return;
}

pid_t pid = fork();

if (pid == -1) {
    perror("fork");
    return;
}

if (pid == 0) {
    execvp(args[0], args);
    perror(args[0]);
    _exit(127);
}

int status;
pid_t result;

do {
    result = waitpid(pid, &status, 0);
} while (result == -1 && errno == EINTR);

if (result == -1) {
    perror("waitpid");
} else if (WIFEXITED(status) &&
           WEXITSTATUS(status) != 0) {
    printf("Command exited with status %d.\n",
           WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("Command terminated by signal %d.\n",
           WTERMSIG(status));
}
}

int main(void)
{
    char input[INPUT_SIZE];

    printf("Q21 - Integrated Shell\n");
    printf("Commands: pwd, cd DIRECTORY, jobs,\n");

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-F Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 integrated_shell.c *
int main(void)
{
    char input[INPUT_SIZE];

    printf("Q21 - Integrated Shell\n");
    printf("Commands: pwd, cd DIRECTORY, jobs,\n");
    printf("        echo TEXT, sleep NUMBER, exit\n");
    printf("Add & for background execution.\n");

    while (1) {
        update_jobs();

        printf("\nq21> ");
        fflush(stdout);

        if (!fgets(input, sizeof(input), stdin))
            break;

        if (!strchr(input, '\n') && !feof(stdin)) {
            int ch;

            while ((ch = getchar()) != '\n' && ch != EOF)
                ;

            fprintf(stderr, "Error: Input too long.\n");
            continue;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        execute_command(input);
    }

    for (int i = 0; i < MAX_JOBS; i++) {
        if (jobs[i].active)
            waitpid(jobs[i].pid, NULL, 0);
    }

    printf("Integrated shell exited.\n");
    return 0;
}
```

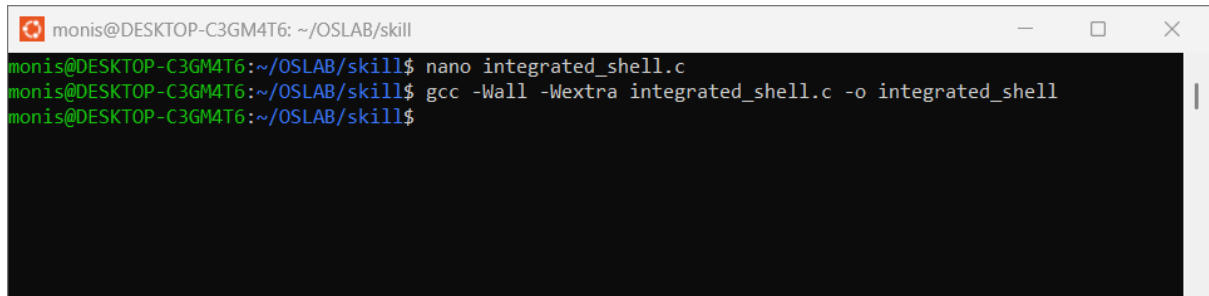
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo

Save with Ctrl + O, Enter, then Ctrl + X.

```
Write to File: integrated_shell.c
^G Help    M-D DOS Format  M-A Append    M-B Backup File ^T Browse  
^C Cancel  M-M Mac Format  M-P Prepend   ^O Discard buffer
```

Compile:

gcc -Wall -Wextra integrated_shell.c -o integrated_shell

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The prompt is 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'. The first command entered is 'nano integrated_shell.c'. The second command is 'gcc -Wall -Wextra integrated_shell.c -o integrated_shell'. The prompt returns after each command.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano integrated_shell.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra integrated_shell.c -o integrated_shell
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 3 — Test end-to-end execution

Run:

`./integrated_shell`

Enter the following commands one at a time:

`pwd`

`echo OSSP Q21 Integration Test`

`cd /tmp`

`pwd`

`sleep 30 &`

`jobs`

`invalidcommand`

`cd /folder_that_does_not_exist`

`jobs`

`exit`

```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./integrated_shell
Q21 - Integrated Shell
Commands: pwd, cd DIRECTORY, jobs,
         echo TEXT, sleep NUMBER, exit
Add & for background execution.

q21> pwd
/home/monis/OSLAB/skill

q21> echo OSSP Q21 Integration Test
OSSP Q21 Integration Test

q21> cd /tmp

q21> pwd
/tmp

q21> sleep 30 &
[1] Background PID=1856

q21> jobs

ID      PID      COMMAND
1       1856     sleep 30 &

q21> invalidcommand
invalidcommand: No such file or directory
Command exited with status 127.

q21> cd/Q
cd/Q: No such file or directory
Command exited with status 127.

[1] Job completed: sleep 30 &

q21> cd /folder_that_does_not_exist
cd: No such file or directory

q21> jobs

ID      PID      COMMAND
No active background jobs.

q21> exit
Integrated shell exited.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Create error_handling.c

Run:

nano error_handling.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano error_handling.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 error_handling.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>

#define INPUT_SIZE 256
#define MAX_ARGS 20

static FILE *log_file;

static void report_error(const char *format, ...)
{
    va_list args;

    va_start(args, format);
    fprintf(stderr, "ERROR: ");
    vfprintf(stderr, format, args);
    fprintf(stderr, "\n");
    va_end(args);

    if (log_file) {
        va_start(args, format);
        fprintf(log_file, "ERROR: ");
        vfprintf(log_file, format, args);
        fprintf(log_file, "\n");
        va_end(args);

        fflush(log_file);
    }
}

static int parse_command(char *input, char *args[])
{
    int count = 0;
    char *token = strtok(input, " \t");

    while (token) {
        if (count >= MAX_ARGS - 1) {
            report_error("Too many arguments.");
        }
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;
    return count;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 error_handling.c *

    return -1;
}

if (strcmp(token, "|") == 0 ||
    strcmp(token, "<") == 0 ||
    strcmp(token, ">") == 0 ||
    strcmp(token, "&") == 0) {
    report_error("Unsupported syntax: %s", token);
    return -1;
}

args[count++] = token;
token = strtok(NULL, " \t");
}

args[count] = NULL;
return count;
}

static void run_command(char *args[])
{
    pid_t pid = fork();

    if (pid == -1) {
        report_error("fork failed: %s", strerror(errno));
        return;
    }

    if (pid == 0) {
        execvp(args[0], args);

        fprintf(stderr, "ERROR: %s: %s\n",
            args[0], strerror(errno));

        _exit(127);
    }

    int status;
    pid_t result;

    do {
        result = waitpid(pid, &status, 0);
    } while (result == -1 && errno == EINTR);

    if (result == -1) {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 error_handling.c *
    report_error("waitpid failed: %s",
                  strerror(errno));
} else if (WIFEXITED(status)) {
    int exit_code = WEXITSTATUS(status);

    if (exit_code != 0)
        report_error("Command '%s' failed with exit status %d.",
                      args[0], exit_code);
    else
        printf("Command executed successfully.\n");
} else if (WIFSIGNALED(status)) {
    report_error("Command '%s' terminated by signal %d.",
                  args[0], WTERMSIG(status));
}
}

int main(void)
{
    char input[INPUT_SIZE];
    char *args[MAX_ARGS];

    log_file = fopen("q21_errors.log", "a");

    if (!log_file) {
        perror("Cannot open log file");
        return 1;
    }

    printf("Q21 - Runtime Error Handling\n");
    printf("Commands: pwd, ls, echo TEXT, cat FILE, exit\n");
    printf("Failures are logged in q21_errors.log.\n");

    while (1) {
        printf("\nerror> ");
        fflush(stdout);

        if (!fgets(input, sizeof(input), stdin)) {
            if (ferror(stdin))
                report_error("Input failed: %s",
                              strerror(errno));

            break;
        }

        if (!strchr(input, '\n') && !feof(stdin)) {
            int ch;
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 error_handling.c *

        strerror(errno));
    }
    break;
}

if (!strchr(input, '\n') && !feof(stdin)) {
    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF)
        ;

    report_error("Input exceeds %d characters.",
        INPUT_SIZE - 2);
    continue;
}

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;

if (input[0] == '\0') {
    report_error("Empty command.");
    continue;
}

int count = parse_command(input, args);

if (count < 0)
    continue;

if (count == 0) {
    report_error("Missing command.");
    continue;
}

run_command(args);
}

if (fclose(log_file) == EOF)
    perror("Closing log file");

printf("Error handling program exited normally.\n");
return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo

Write to File: error_handling.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel     M-M Mac Format M-P Prepend  ^O Discard buffer
```

Save and compile:

gcc -Wall -Wextra error_handling.c -o error_handling

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill$ nano error_handling.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra error_handling.c -o error_handling
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5 — Test error scenarios

Run:

`./error_handling`

Enter these commands one at a time:

`pwd`

`invalidcommand`

`cat missing_file.txt`

`echo hello | wc`

`>`

`echo Recovery successful`

`exit`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill$ ./error_handling
Q21 - Runtime Error Handling
Commands: pwd, ls, echo TEXT, cat FILE, exit
Failures are logged in q21_errors.log.

error> pwd
/home/monis/OSLAB/skill
Command executed successfully.

error> invalidcommand
ERROR: invalidcommand: No such file or directory
ERROR: Command 'invalidcommand' failed with exit status 127.

error> cat missing_file.txt
cat: missing_file.txt: No such file or directory
ERROR: Command 'cat' failed with exit status 1.

error> echo hello | wc
ERROR: Unsupported syntax: |

error> echo Recovery successful
Recovery successful
Command executed successfully.

error> exit
Error handling program exited normally.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

After exiting, view the error log:

`cat q21_errors.log`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat q21_errors.log
ERROR: Command 'invalidcommand' failed with exit status 127.
ERROR: Command 'cat' failed with exit status 1.
ERROR: Unsupported syntax: |
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```